

Benchmark reali, modelli esistenti.

Le primitive di performance T1–T3 (timeline, plan_run, trajectories_run) misurate con due modelli in produzione su Atlas Cloud. Nessun numero sintetico: turni del modello, token riportati dall'API, wall-clock e verdetto *fisico* su disco.

seq · 7 turni



traj · 4 turni



deepseek-ai/deepseek-v4-pro

kwaipilot/kat-coder-pro-v2.5

8/8 run OK post-fix

Due tasse da misurare, un verdetto che non si discute

Bench A — la tassa dei round-trip

Creare 6 file con contenuto esatto. **single**: una `fs_write` per file (plan_run nascosto al modello). **plan**: tutto in **UNA** chiamata `plan_run` transazionale.

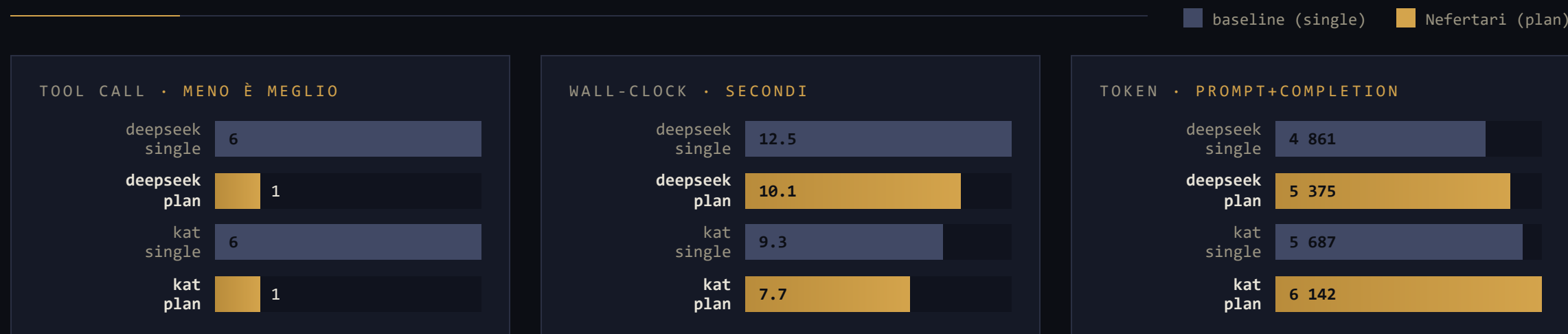
Bench B — la tassa del percorso unico

Trovare quale di 3 tagline passa un test di accettazione (`grep`). **seq**: prova-verifica-riprova, una alla volta. **traj**: **UNA** `trajectories_run` ×3 fork paralleli + promote del vincitore.

Regole di onestà

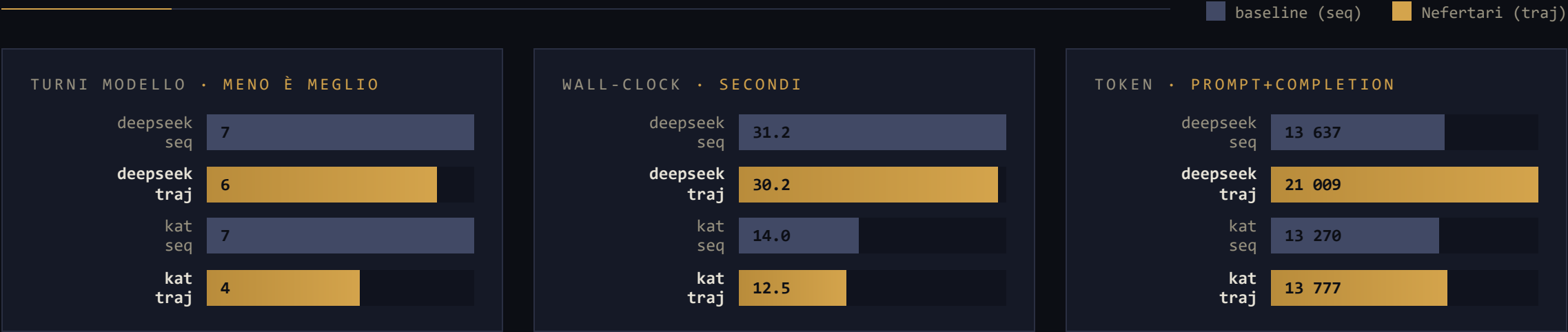
- **Successo fisico**: si verificano i file su disco, mai le parole del modello.
- **Token reali**: campo `usage` dell'API, prompt + completion.
- **Tool nascosti per modalità**: rimossi dalla lista tool; se allucinati, guard che risponde "not available".
- **Ambiente fresco per run**: workspace, `NEFERTARI_HOME` e server MCP nuovi ogni volta.
- **Enforcer Landlock attivo** sulle shell reversibili, broker su ogni azione.

plan_run: 6× meno chiamate, wall più basso



Lettura onesta: entrambi i modelli sanno già batchare 6 `fs_write` parallele in un turno solo (2 turni anche in single) — la tasa dei round-trip sui batch piatti è in gran parte mitigata dal parallel tool calling, e i token sono $\approx$ pari. Il valore di `plan_run` qui è **1 chiamata invece di 6**, wall -20%, e soprattutto la **transazionalità**: uno step fallito = rollback atomico dell'intero piano.

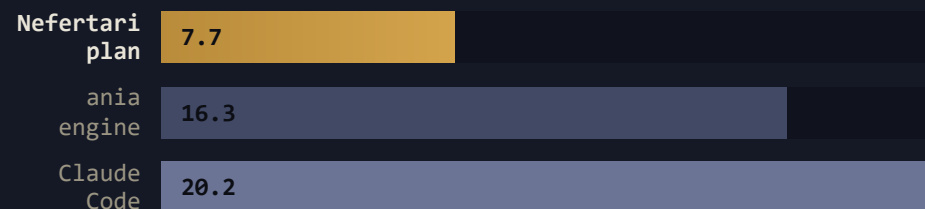
trajectories_run: costo costante dove il sequenziale paga l'ordine



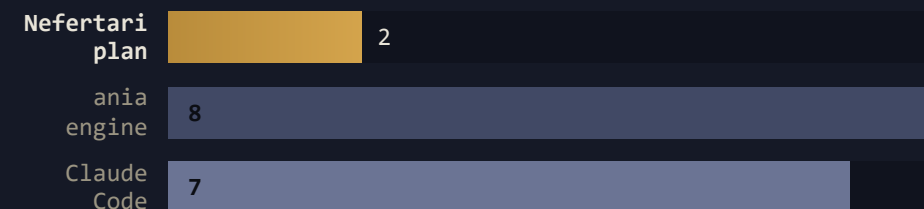
ania engine e Claude Code sugli stessi task

■ Nefertari (kat) ■ ania engine (kat) ■ Claude Code (Fable 5)

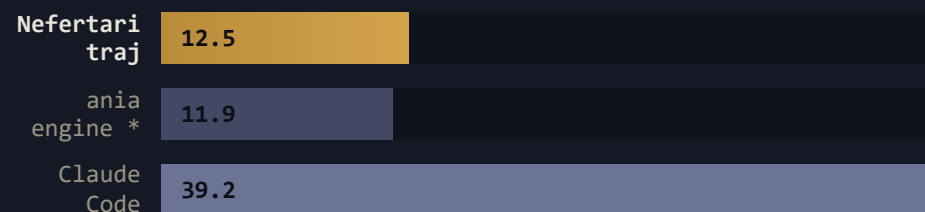
TASK A • 6 FILE — WALL-CLOCK • SECONDI



TASK A — TURNI MODELLO



TASK B • TAGLINE — WALL-CLOCK • SECONDI



TASK B — TURNI MODELLO

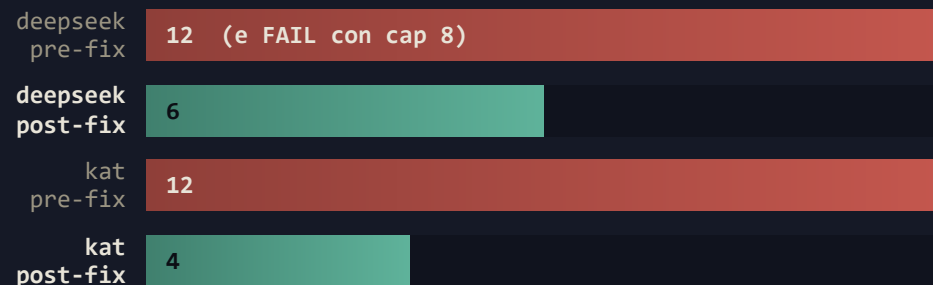


Tutti i run: successo **fisico** su disco. ania gira sullo **stesso modello** del bench (kat-coder) → confronto harness-vs-harness pulito; esegue 1 tool per step, senza batching. * su B ania ha dedotto il vincitore per eliminazione, saltando il grep finale. Claude Code (Fable 5, headless, allow-list) è un harness generalista: token processati 73k (A) / 255k (B), di cui 49k / 230k da cache — costo \$0.49 / \$0.69 contro centesimi su Atlas. Codex CLI: **n/d** in questa sessione (log ChatGPT scaduto, 401).

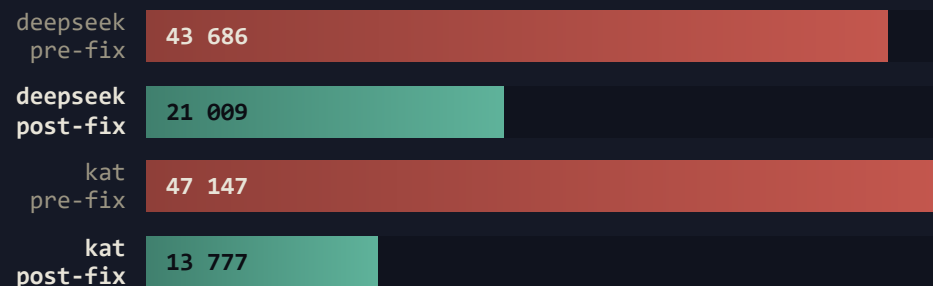
Checkpoint di una directory vuota → ENOENT

pre-fix post-fix

TURNI • MODALITÀ TRAJ



TOKEN • MODALITÀ TRAJ



Cosa succedeva

`checkpoint()` e `fork()` creavano tree/ solo copiando file: workspace vuoto = ENOENT sul manifest. **Entrambi i modelli**, indipendentemente, hanno bruciato ~8 turni (ls, timeline_list, mkdir...) prima di trovare da soli il workaround: un file placeholder e ri-checkpoint.

Il fix (una riga ×2) + regressione

`mkdirSync(tree, {recursive:true})` prima della copia, in checkpoint e fork. Test nuovo: dir vuota → checkpoint / fork / promote. Suite **9/9 PASS**. Commit 3a845c4. È esattamente ciò che un benchmark reale deve produrre.

Cosa dicono davvero questi numeri

Dove le primitive vincono

- **traj** ha costo costante; **seq** scala linearmente coi tentativi. Con K candidati e eval costose (test suite reali) il divario esplode a favore dei fork paralleli.
- **plan**: 1 chiamata invece di N, wall più basso, e rollback atomico che nessuna sequenza di write singole può dare.
- Ogni fork ha **enforcement gratis**: le shell degli step girano con cwd nel fork, Landlock confina le scritture lì.
- Il journal registra tutto: winner, fork, piano, classe — auditabilità che il modo "flat" non produce.

Dove essere cauti

- Il **parallel tool calling** mitiga già la tasa dei round-trip sui batch piatti e indipendenti: lì plan_run vince di poco.
- traj costa **più token** quando il sequenziale avrebbe indovinato presto (deepseek: 21k vs 13.6k) — è un'assicurazione, si paga.
- Gli argomenti JSON annidati di plan/traj sono **fragili per alcuni modelli**: deepseek in run precedenti ha emesso markup DSML e args vuoti (2 retry) — varianza, non determinismo. In questi run finali: primo colpo per entrambi.
- Campione piccolo: 1 run per cella, 2 modelli, task brevi. Direzione chiara, grandezze indicative.

8 run, 8 successi fisici

BENCH	MODALITÀ	MODELLO	TURNI	TOOL CALL	TOKEN	WALL	ESITO FISICO
A · 6 file	single	deepseek-v4-pro	2	6	4 861	12.5 s	OK
	plan	deepseek-v4-pro	2	1	5 375	10.1 s	OK
	single	kat-coder-pro-v2.5	2	6	5 687	9.3 s	OK
	plan	kat-coder-pro-v2.5	2	1	6 142	7.7 s	OK
B · tagline	seq	deepseek-v4-pro	7	6	13 637	31.2 s	OK
	traj	deepseek-v4-pro	6	6	21 009	30.2 s	OK
	seq	kat-coder-pro-v2.5	7	6	13 270	14.0 s	OK
	traj	kat-coder-pro-v2.5	4	3	13 777	12.5 s	OK

Run storici pre-fix (bug dir-vuota): deepseek traj **FAIL** con cap 8 turni, poi 12 turni / 43 686 tok con cap 12; kat traj 12 / 47 147 tok.
Contesto: il test BRAND end-to-end su processo Fortuna (deepseek, T1+T2+T3 + gate umano sul publish) resta **7/7 verifiche fisiche**.

La catena continua

T4 · world

Stato macchina tipizzato + delta-dall'ultima-sessione ricostruito dal journal: il modello riparte senza ri-esplorare.

T5 · typed state

Stato di sistema strutturato interrogabile, al posto di parse di output shell.

T6 · postcondition

Assert dichiarative post-azione: il piano fallisce subito, non tre step dopo.

preflight(action)

{class, will_gate, undo_via} senza eseguire — la storia robotica: sapere prima di muovere.

Bench: prossima cella

Team-variant: 3 modelli diversi, una traiettoria a testa sullo stesso checkpoint, un reviewer decide il promote.

Campione più largo

N run per cella per medie e varianza; task più lunghi dove le tasse compongono.